

Project Architecture & Reference

This document is the source of truth for the mobile app's tech stack, folder structure, database schema, and API surface. Keep it in the repo root (`/ARCHITECTURE.md`) so Cursor / Claude Code / any AI assistant can read it as full project context.

1. Tech Stack Summary

Layer	Tool	Purpose
Mobile frontend	Flutter (Dart)	iOS + Android single codebase
State management	Riverpod 2.x	Reactive state, dependency injection
Routing	go_router	Declarative routing with deep links
HTTP client	Dio	API calls with interceptors
Local storage	flutter_secure_storage + shared_preferences	JWT/secrets + cache
Backend	Supabase	Postgres DB, Auth, Realtime, Edge Functions
Image hosting	Cloudinary	Profile pics, verification photos, chat media
SMS / OTP	MSG91	OTP delivery only (auth state stays in Supabase)
Payments	Razorpay	Coin purchases (male side) + payout processing (admin)
Push notifications	Firebase Cloud Messaging (FCM)	Chat requests, payment alerts, payout updates
Error tracking	Sentry	Production crash + error reporting (recommended)
API documentation	Swagger / OpenAPI 3.0	All Edge Function endpoints documented
Realtime updates	Supabase Realtime	Live chat request notifications to female

Key architectural decisions

- **Supabase Auth is the source of truth for sessions.** MSG91 is just the SMS pipeline.
 - **All sensitive data (user info, sessions, payouts, ratings) lives in Supabase** (Mumbai region — `ap-south-1`) for DPDP Act compliance.
 - **Cloudinary stores media** (signed uploads from the client, signature generated by an Edge Function so the API secret never touches the device).
 - **Edge Functions handle business logic** that can't be a simple table operation (OTP send/verify, payment verification, payout requests, chat ratings).
 - **Direct PostgREST (Supabase auto-generated REST) is used for simple CRUD** with RLS policies enforcing security.
-

2. Repository Layout (top level)

```
my-app/
├── ARCHITECTURE.md           # this file
├── README.md                 # how to run locally
├── mobile/                   # Flutter app
│   └── ... (see Section 3)
├── supabase/                 # Backend (DB + Edge Functions)
│   └── ... (see Section 4)
├── docs/
│   ├── api-reference.md      # mirrors Section 6
│   ├── auth-flow.md          # detailed auth diagrams
│   ├── data-model.md         # ER diagram + schema notes
│   └── swagger.yaml          # OpenAPI spec for Edge Functions
└── .env.example              # template for env vars (never commit real
                               values)
```

3. Flutter Project Structure (`mobile/`)

Feature-based layout with clear separation between data, domain, and presentation layers.

```

mobile/
├─ pubspec.yaml
├─ analysis_options.yaml      # lint rules (flutter_lints)
├─ .env                       # local env (gitignored)
├─ android/
├─ ios/
├─ assets/
│   ├─ images/                # static images, app logo, default avatars
│   ├─ icons/                 # custom icons
│   ├─ illustrations/        # empty-state and error illustrations
│   └─ fonts/                 # custom font files if any
├─ lib/
│   ├─ main.dart              # entry point
│   ├─ app.dart               # MaterialApp with theme + router
│   │
│   ├─ core/                  # app-wide infrastructure
│   │   ├─ config/
│   │   │   ├─ env.dart       # loads .env via flutter_dotenv
│   │   │   ├─ app_config.dart # app-wide constants (API base URL, timeouts)
│   │   │   └─ flavors.dart   # dev/staging/prod config
│   │   └─ theme/
│   │       ├─ app_theme.dart  # ThemeData (light/dark)
│   │       ├─ app_colors.dart # color palette (pink, blue, neutrals)
│   │       └─ app_typography.dart # text styles
│   │           └─ app_spacing.dart # spacing constants (s, m, l, xl)
│   │   └─ routing/
│   │       ├─ app_router.dart # go_router configuration
│   │       ├─ route_names.dart # route constants
│   │       └─ auth_guard.dart  # redirects unauthenticated users
│   │   └─ network/
│   │       ├─ api_client.dart  # Dio instance + base config
│   │       ├─ api_endpoints.dart # URL constants
│   │       ├─ auth_interceptor.dart # injects Bearer token
│   │       ├─ error_interceptor.dart # converts errors to AppException
│   │       └─ api_exception.dart # custom exception types
│   │   └─ storage/
│   │       ├─ secure_storage.dart # token storage wrapper
│   │       ├─ cache_storage.dart  # shared_prefs wrapper
│   │       └─ storage_keys.dart   # key constants
│   │   └─ services/
│   │       ├─ supabase_service.dart # Supabase client init + auth state
│   │       ├─ cloundinary_service.dart # signed image upload
│   │       ├─ notification_service.dart # FCM setup + handlers
│   │       ├─ razorpay_service.dart # payment integration
│   │       ├─ realtime_service.dart # Supabase Realtime subscriptions
│   │       └─ permission_service.dart # camera, gallery, notification perms

```

- └─ utils/
 - └─ validators.dart # form validators
 - └─ formatters.dart # currency, date, phone formatting
 - └─ date_utils.dart # relative time ("2h ago"), etc.
 - └─ logger.dart # structured logging
 - └─ extensions/
 - └─ context_ext.dart
 - └─ string_ext.dart
 - └─ num_ext.dart
- └─ shared/ # reusable across features
 - └─ widgets/
 - └─ buttons/
 - └─ primary_button.dart
 - └─ secondary_button.dart
 - └─ icon_button_rounded.dart
 - └─ inputs/
 - └─ app_text_field.dart
 - └─ password_field.dart
 - └─ phone_field.dart
 - └─ otp_input.dart
 - └─ cards/
 - └─ stat_card.dart
 - └─ info_card.dart
 - └─ loaders/
 - └─ full_screen_loader.dart
 - └─ shimmer_card.dart
 - └─ button_loader.dart
 - └─ error_views/
 - └─ error_view.dart
 - └─ offline_view.dart
 - └─ empty_states/
 - └─ empty_state_view.dart
 - └─ dialogs/
 - └─ confirmation_dialog.dart
 - └─ bottom_sheet_picker.dart
 - └─ status_indicator.dart # online/offline/available dot
 - └─ avatar.dart # profile pic with fallback
 - └─ app_bar.dart # standard app bar
 - └─ models/ # shared data models
 - └─ user.dart
 - └─ female_profile.dart
 - └─ male_profile.dart
 - └─ chat.dart
 - └─ chat_request.dart
 - └─ transaction.dart
 - └─ payout.dart

```

└─ notification.dart
└─ coin_package.dart
└─ rating.dart
└─ api_response.dart
└─ providers/          # global Riverpod providers
    └─ auth_provider.dart
    └─ current_user_provider.dart
    └─ connectivity_provider.dart

└─ features/          # one folder per feature
    └─ splash/
        └─ presentation/
            └─ splash_screen.dart

    └─ auth/
        └─ data/
            └─ auth_api.dart          # API calls
            └─ auth_repository.dart   # business logic + caching
        └─ domain/
            └─ auth_state.dart        # state classes
        └─ presentation/
            └─ providers/
                └─ auth_notifier.dart # Riverpod notifier
            └─ screens/
                └─ account_type_screen.dart
                └─ female_signup_basic_screen.dart
                └─ female_signup_otp_screen.dart
                └─ female_signup_bank_screen.dart
                └─ female_verification_info_screen.dart
                └─ face_capture_screen.dart
                └─ verification_submitted_screen.dart
                └─ female_login_phone_screen.dart
                └─ female_login_password_screen.dart
                └─ verification_pending_dialog.dart
                └─ male_signup_screen.dart
                └─ male_signup_otp_screen.dart
                └─ male_onboarding_screen.dart
                └─ male_login_screen.dart
                └─ forgot_password_phone_screen.dart
                └─ forgot_password_otp_screen.dart
                └─ forgot_password_reset_screen.dart
            └─ widgets/
                └─ otp_resend_timer.dart
                └─ face_outline_overlay.dart

    └─ female_home/
        └─ data/

```

```
| | | └─ domain/
| | | └─ presentation/
| | |   └─ providers/
| | |   └─ screens/
| | |     └─ female_home_screen.dart
| | |   └─ widgets/
| | |     └─ availability_toggle_card.dart
| | |     └─ stats_grid.dart
| | |     └─ recent_activity_list.dart
| | |
| | | └─ female_earnings/
| | |   └─ presentation/
| | |     └─ screens/
| | |       └─ earnings_dashboard_screen.dart
| | |       └─ bank_update_screen.dart
| | |     └─ widgets/
| | |       └─ earnings_stats_row.dart
| | |       └─ payout_request_card.dart
| | |       └─ payout_status_banner.dart
| | |       └─ bank_details_card.dart
| | |       └─ chat_history_filter.dart
| | |       └─ payout_confirmation_dialog.dart
| | |
| | | └─ male_home/
| | |   └─ presentation/
| | |     └─ screens/
| | |       └─ male_home_screen.dart
| | |       └─ female_profile_preview_screen.dart
| | |     └─ widgets/
| | |       └─ coins_pill.dart
| | |       └─ favourites_carousel.dart
| | |       └─ female_card.dart
| | |       └─ filter_chips.dart
| | |       └─ search_bar.dart
| | |
| | | └─ male_wallet/
| | |   └─ presentation/
| | |     └─ screens/
| | |       └─ wallet_screen.dart           # parent with slider tabs
| | |       └─ payment_processing_screen.dart
| | |       └─ payment_success_screen.dart
| | |       └─ payment_failed_screen.dart
| | |     └─ widgets/
| | |       └─ wallet_tab_view.dart
| | |       └─ transaction_tab_view.dart
| | |       └─ balance_hero_card.dart
| | |       └─ coin_package_card.dart
```

```

| | | | | purchase_confirmation_dialog.dart
| | | | | insufficient_coins_dialog.dart
| | | | | transaction_row.dart
| | | | |
| | | | |
| | | | | profile/ # shared between male and female
| | | | |   └─ presentation/
| | | | |       │
| | | | |       │ └─ screens/
| | | | |           │
| | | | |           │ └─ profile_screen.dart
| | | | |           │ └─ help_support_screen.dart
| | | | |           │ └─ report_issue_screen.dart
| | | | |           │ └─ about_app_screen.dart
| | | | |           │ └─ settings_screen.dart
| | | | |           │ └─ change_password_screen.dart
| | | | |           │ └─ delete_account_warning_screen.dart
| | | | |           │ └─ delete_account_confirm_screen.dart
| | | | |       └─ widgets/
| | | | |           │
| | | | |           │ └─ profile_menu_row.dart
| | | | |           │ └─ edit_picture_sheet.dart
| | | | |           │ └─ logout_confirmation_dialog.dart
| | | | |
| | | | |
| | | | | └─ chat_request/ # Phase 1: just request popup + rating
| | | | |   └─ presentation/
| | | | |       │
| | | | |       │ └─ screens/
| | | | |           │
| | | | |           │ └─ chat_request_sent_screen.dart # male
| | | | |           │ └─ chat_request_declined_screen.dart # male
| | | | |           │ └─ chat_request_timeout_screen.dart # male
| | | | |           │ └─ queue_position_screen.dart # male (Phase 2 UI
ready)
| | | | |           │ └─ like_dislike_rating_screen.dart # male, post-chat
| | | | |       └─ widgets/
| | | | |           │
| | | | |           │ └─ incoming_request_dialog.dart # female
| | | | |           │ └─ auto_decline_timer.dart
| | | | |
| | | | |
| | | | | └─ chat/ # Phase 2 placeholder
| | | | |   └─ presentation/
| | | | |       │
| | | | |       │ └─ screens/
| | | | |           │
| | | | |           │ └─ active_chat_screen.dart # to be built
| | | | |
| | | | |
| | | | | └─ notifications/
| | | | |   └─ presentation/
| | | | |       │
| | | | |       │ └─ screens/
| | | | |           │
| | | | |           │ └─ notifications_screen.dart
| | | | |
| | | | |
| | | | | └─ block_report/
| | | | |   └─ presentation/
| | | | |       │
| | | | |       │ └─ widgets/
| | | | |           │
| | | | |           │ └─ block_report_sheet.dart

```

```
| | | └─ report_reason_sheet.dart
| | |
| | └─ system/ # offline, errors, updates
| |   └─ presentation/
| |     ├── offline_screen.dart
| |     ├── update_available_screen.dart
| |     ├── maintenance_screen.dart
| |     ├── account_suspended_screen.dart
| |     ├── session_expired_dialog.dart
| |     └─ generic_error_screen.dart
| |
| └─ l10n/ # localization (future-proof)
|   └─ app_en.arb
|
└─ test/ # mirror lib/ structure
  ├── core/
  ├── features/
  └─ widget_test.dart
```

Recommended `pubspec.yaml` dependencies

yaml

```
dependencies:
  flutter:
    sdk: flutter

  # State management
  flutter_riverpod: ^2.5.1
  riverpod_annotation: ^2.3.5

  # Routing
  go_router: ^14.0.0

  # Network
  dio: ^5.4.0
  pretty_dio_logger: ^1.4.0      # dev only

  # Backend
  supabase_flutter: ^2.5.0

  # Storage
  flutter_secure_storage: ^9.0.0
  shared_preferences: ^2.2.2

  # Media
  image_picker: ^1.0.7          # for camera/gallery
  cached_network_image: ^3.3.1
  image_cropper: ^7.0.0         # optional, for profile pic editing

  # Camera (for face verification)
  camera: ^0.10.5

  # Payments
  razorpay_flutter: ^1.3.6

  # Notifications
  firebase_core: ^2.27.0
  firebase_messaging: ^14.7.20

  # Utilities
  flutter_dotenv: ^5.1.0
  intl: ^0.19.0                 # date / number formatting
  freezed_annotation: ^2.4.1
  json_annotation: ^4.8.1
  flutter_local_notifications: ^17.0.0
  permission_handler: ^11.3.0
  url_launcher: ^6.2.5
  package_info_plus: ^6.0.0
```

```

connectivity_plus: ^6.0.0

# UI helpers
shimmer: ^3.0.0
lottie: ^3.1.0           # for nice animated illustrations
fluttertoast: ^8.2.4

dev_dependencies:
  flutter_lints: ^4.0.0
  build_runner: ^2.4.8
  riverpod_generator: ^2.4.0
  freezed: ^2.4.7
  json_serializable: ^6.7.1

```

4. Supabase Backend Structure (`supabase/`)

```

supabase/
├─ config.toml           # Supabase project config
├─ seed.sql              # seed data for dev/testing
├─
├─ migrations/          # SQL migrations (timestamped)
│   ├─ 20260101000001_create_users.sql
│   ├─ 20260101000002_create_female_profiles.sql
│   ├─ 20260101000003_create_male_profiles.sql
│   ├─ 20260101000004_create_otp_codes.sql
│   ├─ 20260101000005_create_verification.sql
│   ├─ 20260101000006_create_bank_details.sql
│   ├─ 20260101000007_create_chat_requests.sql
│   ├─ 20260101000008_create_chats.sql
│   ├─ 20260101000009_create_chat_messages.sql      # Phase 2
│   ├─ 20260101000010_create_ratings.sql
│   ├─ 20260101000011_create_favourites.sql
│   ├─ 20260101000012_create_coin_packages.sql
│   ├─ 20260101000013_create_transactions.sql
│   ├─ 20260101000014_create_payouts.sql
│   ├─ 20260101000015_create_notifications.sql
│   ├─ 20260101000016_create_blocks_reports.sql
│   ├─ 20260101000017_create_indexes.sql
│   ├─ 20260101000018_create_views.sql              # aggregations
│   ├─ 20260101000019_create_functions.sql          # PG functions / triggers
│   └─ 20260101000020_create_rls_policies.sql
├─
└─ functions/           # Edge Functions (Deno + TypeScript)
    └─ _shared/

```

```
|   ├── cors.ts           # CORS headers
|   ├── auth.ts           # JWT verification helper
|   ├── supabase.ts       # service-role client init
|   ├── msg91.ts          # MSG91 OTP send
|   ├── cloudinary.ts     # signed URL generation
|   ├── razorpay.ts       # order creation + signature verify
|   ├── fcm.ts            # push notification sender
|   ├── otp_utils.ts      # generate, hash, verify OTP
|   └── types.ts          # shared TypeScript types
|
|── auth-otp-send/
|   └── index.ts           # POST /functions/v1/auth-otp-send
|── auth-otp-verify/
|   └── index.ts
|── auth-signup-complete/
|   └── index.ts
|── auth-login-check/
|   └── index.ts           # checks verification status, routes accordingly
|── auth-forgot-password-send/
|   └── index.ts
|── auth-forgot-password-verify/
|   └── index.ts
|── auth-forgot-password-reset/
|   └── index.ts
|
|── upload-sign/
|   └── index.ts           # Cloudinary upload signature
|
|── female-payout-request/
|   └── index.ts           # validates & creates payout row
|
|── male-payment-create/
|   └── index.ts           # creates Razorpay order
|── male-payment-verify/
|   └── index.ts           # verifies signature, credits coins
|
|── male-chat-request-send/
|   └── index.ts           # creates chat_request, checks coin balance
|── male-chat-request-cancel/
|   └── index.ts
|── female-chat-request-respond/
|   └── index.ts           # accept or decline
|
|── chat-rate/
|   └── index.ts           # records like/dislike, updates female rating
|
|── block-user/
```

```
|   └─ index.ts
└─ report-user/
    └─ index.ts
|
└─ notification-fcm-token-register/
    └─ index.ts
└─ support-report-issue/
    └─ index.ts
```

5. Database Schema (Supabase / Postgres)

Concise overview. Full DDL lives in the migration files.

users (extends Supabase **auth.users**)

Column	Type	Notes
id	uuid (PK, FK → auth.users.id)	Same as Supabase auth user id
username	text	display name
gender	enum ((male), (female))	role
age	int	
phone	text (unique)	E.164 format
profile_pic_url	text (nullable)	Cloudinary URL
is_active	boolean (default true)	soft delete flag
is_suspended	boolean (default false)	admin can suspend
last_active_at	timestampz	updated on every session refresh
created_at	timestampz	
updated_at	timestampz	

female_profiles

Column	Type	Notes
user_id	uuid (PK, FK → users.id)	

Column	Type	Notes
verification_status	enum (pending, verified, rejected)	default pending
verification_photo_url	text	Cloudinary URL
verification_submitted_at	timestamptz	
verification_reviewed_at	timestamptz (nullable)	
verification_reviewed_by	uuid (FK → admins.id, nullable)	
is_online	boolean (default false)	toggle state
last_online_at	timestamptz	
total_earnings	numeric(12,2) (default 0)	
available_balance	numeric(12,2) (default 0)	unpaid earnings
claimed_amount	numeric(12,2) (default 0)	completed payouts
requested_amount	numeric(12,2) (default 0)	sum of pending payout requests
total_chats_received	int (default 0)	
total_likes	int (default 0)	
total_dislikes	int (default 0)	
favourited_by_count	int (default 0)	
rating	numeric(3,2) (default 0)	computed from likes/dislikes
bio	text (nullable)	

male_profiles

Column	Type	Notes
user_id	uuid (PK, FK → users.id)	
coin_balance	int (default 0)	current coins
total_coins_purchased	int (default 0)	

Column	Type	Notes
total_coins_spent	int (default 0)	
total_inr_spent	numeric(12,2) (default 0)	
total_purchase_count	int (default 0)	
total_chats_made	int (default 0)	

otp_codes

Column	Type	Notes
id	uuid (PK)	
phone	text	
code_hash	text	bcrypt hash of OTP
purpose	enum ((signup),(login),(forgot_password))	
attempts	int (default 0)	rate limiting
expires_at	timestampz	typically 5 min
consumed_at	timestampz (nullable)	one-time use
created_at	timestampz	

Index on ((phone, purpose, expires_at desc)) for fast lookup.

bank_details

Column	Type	Notes
user_id	uuid (PK, FK → users.id)	one per female
type	enum ((bank),(upi))	
account_holder_name	text (nullable)	
account_number	text (nullable, encrypted)	
ifsc_code	text (nullable)	
upi_id	text (nullable)	

Column	Type	Notes
updated_at	timestamptz	

chat_requests

Column	Type	Notes
id	uuid (PK)	
male_id	uuid (FK → users.id)	
female_id	uuid (FK → users.id)	
status	enum (pending, accepted, declined, cancelled, timeout)	
queue_position	int (nullable)	for Phase 2 queue
created_at	timestamptz	
responded_at	timestamptz (nullable)	

chats

Column	Type	Notes
id	uuid (PK)	this is the Chat_ID shown to admins
request_id	uuid (FK → chat_requests.id, unique)	
male_id	uuid (FK → users.id)	
female_id	uuid (FK → users.id)	
started_at	timestamptz	
ended_at	timestamptz (nullable)	
duration_seconds	int (nullable, generated)	
coins_consumed	int (default 0)	
inr_credited_to_female	numeric(10,2) (default 0)	based on 40/60 split

Column	Type	Notes
ended_by	enum ((male), (female), (system_timeout), nullable)	

chat_messages (Phase 2)

Column	Type	Notes
id	uuid (PK)	
chat_id	uuid (FK → chats.id)	
sender_id	uuid (FK → users.id)	
type	enum ((text), (image), (video))	
content	text	for text type; URL for media
created_at	timestampz	

ratings

Column	Type	Notes
id	uuid (PK)	
chat_id	uuid (FK → chats.id, unique)	one rating per chat
male_id	uuid (FK → users.id)	
female_id	uuid (FK → users.id)	
rating	enum ((like), (dislike), (skipped))	
comment	text (nullable)	optional feedback
created_at	timestampz	

A DB trigger on this table updates `female_profiles.total_likes / total_dislikes / rating` automatically.

favourites

Column	Type	Notes
male_id	uuid (FK → users.id)	
female_id	uuid (FK → users.id)	
created_at	timestamptz	
PRIMARY KEY ((male_id), (female_id))		

coin_packages

Column	Type	Notes
id	uuid (PK)	
coins	int	
price_inr	numeric(8,2)	
label	text (nullable)	e.g., "Best Value"
display_order	int	
is_active	boolean (default true)	

transactions

Column	Type	Notes
id	uuid (PK)	
user_id	uuid (FK → users.id)	the male
type	enum ((purchase), (spend))	
coins	int	positive for purchase, negative for spend
amount_inr	numeric(10,2) (nullable)	for purchases only
status	enum ((pending), (success), (failed))	purchases only

Column	Type	Notes
<code>chat_id</code>	uuid (FK → chats.id, nullable)	for spends
<code>razorpay_order_id</code>	text (nullable)	
<code>razorpay_payment_id</code>	text (nullable)	
<code>razorpay_signature</code>	text (nullable)	
<code>created_at</code>	timestamptz	

payouts

Column	Type	Notes
<code>id</code>	uuid (PK)	
<code>female_id</code>	uuid (FK → users.id)	
<code>amount_inr</code>	numeric(10,2)	
<code>status</code>	enum ((<code>pending</code>), (<code>approved</code>), (<code>completed</code>), (<code>rejected</code>))	
<code>bank_snapshot</code>	jsonb	snapshot of bank details at request time
<code>requested_at</code>	timestamptz	
<code>approved_at</code>	timestamptz (nullable)	
<code>completed_at</code>	timestamptz (nullable)	
<code>rejected_at</code>	timestamptz (nullable)	
<code>rejection_reason</code>	text (nullable)	
<code>processed_by</code>	uuid (FK → admins.id, nullable)	
<code>utr_number</code>	text (nullable)	bank transfer reference

Constraint: at most one row per `female_id` with status IN ((`pending`), (`approved`)).

notifications

Column	Type	Notes
id	uuid (PK)	
user_id	uuid (FK → users.id)	
type	enum ((chat_request, payment, payout, verification, system))	
title	text	
body	text	
data	jsonb	navigation payload
is_read	boolean (default false)	
created_at	timestampz	

fcm_tokens

Column	Type	Notes
id	uuid (PK)	
user_id	uuid (FK → users.id)	
token	text (unique)	FCM device token
platform	enum ((ios, android))	
last_used_at	timestampz	

blocks

Column	Type	Notes
blocker_id	uuid (FK → users.id)	
blocked_id	uuid (FK → users.id)	
created_at	timestampz	

Column	Type	Notes
PRIMARY KEY ((blocker_id), (blocked_id))		

reports

Column	Type	Notes
id	uuid (PK)	
reporter_id	uuid (FK → users.id)	
reported_id	uuid (FK → users.id)	
reason	text	
comment	text (nullable)	
chat_id	uuid (FK → chats.id, nullable)	
status	enum ((open), (resolved), (dismissed))	
created_at	timestampz	

admins

Column	Type	Notes
id	uuid (PK)	
email	text (unique)	
name	text	
role	enum ((super_admin), (admin), (moderator))	
is_active	boolean (default true)	

admin_audit_log

Every Approve / Reject / Complete action on payouts and verifications goes here.

Column	Type	Notes
<code>id</code>	uuid (PK)	
<code>admin_id</code>	uuid (FK → admins.id)	
<code>action</code>	text	e.g., <code>payout.approve</code> , <code>verification.reject</code>
<code>target_table</code>	text	
<code>target_id</code>	uuid	
<code>metadata</code>	jsonb	
<code>created_at</code>	timestampz	

6. API Reference

Two API surfaces:

- **PostgREST (auto-generated by Supabase):** direct CRUD on tables, secured by RLS. Used for reads and simple writes (e.g., toggle online, list transactions). No need to document each — Supabase generates an OpenAPI spec automatically.
- **Edge Functions (custom logic):** every endpoint below. Documented in `docs/swagger.yaml`.

Base URL pattern: `https://<project>.supabase.co/functions/v1/<function-name>`

All endpoints require `Authorization: Bearer <jwt>` except the OTP send/verify endpoints.

6.1 Auth & OTP

POST /auth-otp-send

Initiates OTP send. Creates a row in `otp_codes`, calls MSG91 API to deliver.

Request:

```
json
{
  "phone": "+919876543210",
  "purpose": "signup"
}
```

Response (200):

```
json
{
  "success": true,
  "expires_in": 300,
  "message": "OTP sent successfully"
}
```

Errors: 429 (rate limit), 400 (invalid phone), 500 (MSG91 failure).

POST /auth-otp-verify

Verifies the OTP and returns a Supabase session (for signup/login purposes).

Request:

```
json
{
  "phone": "+919876543210",
  "code": "123456",
  "purpose": "signup"
}
```

Response (200):

```
json
{
  "success": true,
  "access_token": "...",
  "refresh_token": "...",
  "user_exists": false
}
```

POST /auth-signup-complete

After OTP verify, completes user creation with profile details.

Request:

```
json
```

```
{
  "phone": "+919876543210",
  "name": "Priya",
  "age": 24,
  "gender": "female",
  "password": "...
}
```

Response (200):

```
json
{
  "user_id": "uuid",
  "access_token": "...",
  "refresh_token": "...",
  "next_step": "bank_details"
}
```

POST /auth-login-check

Checks female verification status before requesting password. Returns routing instruction.

Request:

```
json
{
  "phone": "+919876543210"
}
```

Response (200):

```
json
{
  "user_exists": true,
  "verification_status": "verified", // or "pending", "missing"
  "next_step": "password"           // or "verification_pending_popup", "face_capt
}
```

POST /auth-forgot-password-send

Same shape as /auth-otp-send with purpose: "forgot_password".

POST /auth-forgot-password-verify

Verifies OTP for password reset, returns a short-lived reset token.

POST /auth-forgot-password-reset

Uses reset token to set new password.

Request:

```
json
{
  "reset_token": "...",
  "new_password": "..."
}
```

POST /auth/logout (PostgREST)

Standard Supabase Auth signOut.

6.2 User profile (common)

These use PostgREST directly with RLS:

Method	Endpoint	Purpose
GET	<code>/rest/v1/users?id=eq.{me}</code>	get my profile
PATCH	<code>/rest/v1/users?id=eq.{me}</code>	update profile
GET	<code>/rest/v1/female_profiles?user_id=eq.{me}</code>	female-specific data
GET	<code>/rest/v1/male_profiles?user_id=eq.{me}</code>	male-specific data

POST /upload-sign

Returns a signed Cloudinary upload signature for the client to upload directly to Cloudinary.

Request:

```
json
{
  "folder": "verification_photos",
  "public_id": "user_<uuid>_verify"
}
```

Response:

```
json
{
  "signature": "...",
  "timestamp": 1715634000,
  "api_key": "...",
  "cloud_name": "...",
  "folder": "verification_photos"
}
```

Client uploads to Cloudinary directly with this signature, then PATCHes the user row with the resulting URL.

POST /change-password (Edge Function)

Validates current password before updating.

DELETE /delete-account (Edge Function)

Multi-step delete with checks (no pending payouts, etc.). Marks user `is_active = false`.

6.3 Female-specific

Availability toggle

PostgREST PATCH on `female_profiles.is_online`.

Home stats

GET on a Postgres view `female_home_stats_view` which returns total earnings, today's earnings, ratings, total chats in one row.

Recent activity

GET on `chats` table filtered by `female_id = me`, ordered by `started_at desc`, limit 3.

Earnings dashboard stats

GET on a view returning all 4 numbers.

POST /female-payout-request

Request a payout for the entire `available_balance`.

Request: `{}` (no body needed, amount = available_balance) Response:

```
json
```

```
{
  "payout_id": "uuid",
  "amount_inr": 4200.00,
  "status": "pending"
}
```

Validates: bank details exist, no other pending payout, balance > 0.

Payout history

PostgREST GET on `payouts` filtered by `female_id = me`.

Bank details

PostgREST GET/PATCH on `bank_details`.

Chat history with filters

PostgREST GET on `chats` with filter `started_at >= today/week_start/month_start`.

6.4 Male-specific

Available females

GET on a view `available_females_view` with filters: search, status (online/available/all), pagination, sort by rating.

Example:

```
GET /rest/v1/available_females_view?
  online_status=in.(online,available)&
  username=ilike.*priya*&
  order=rating.desc&
  limit=20&offset=0
```

Favourites

- GET `/rest/v1/favourites?male_id=eq.{me}&select=*,female:users(*)`
- POST `/rest/v1/favourites` (toggle on)
- DELETE `/rest/v1/favourites?male_id=eq.{me}&female_id=eq.{id}` (toggle off)

Wallet balance

GET on `male_profiles?user_id=eq.{me}` (coin_balance + total_coins_spent).

Coin packages

GET on `coin_packages?is_active=eq.true&order=display_order`.

POST `/male-payment-create`

Creates a Razorpay order for a coin package purchase.

Request:

```
json
{
  "package_id": "uuid"
}
```

Response:

```
json
{
  "razorpay_order_id": "order_xxx",
  "amount_paise": 50000,
  "currency": "INR",
  "key": "rzp_test_xxx"
}
```

POST `/male-payment-verify`

Called after Razorpay returns success. Verifies signature, credits coins.

Request:

```
json
{
  "razorpay_order_id": "order_xxx",
  "razorpay_payment_id": "pay_xxx",
  "razorpay_signature": "...",
}
```

Response:

```
json
{
  "success": true,
  "coins_added": 100,
  "new_balance": 350,
  "transaction_id": "uuid"
}
```

Transactions

PostgREST GET on `transactions` filtered by user and type.

`POST /male-chat-request-send`

Creates a chat request, validates coin balance.

Request:

```
json
{
  "female_id": "uuid"
}
```

Response:

```
json
{
  "request_id": "uuid",
  "status": "pending",
  "queue_position": null
}
```

Errors: 402 (insufficient coins), 404 (female not online), 403 (blocked).

`POST /male-chat-request-cancel`

Cancels a pending request.

Request: `{ "request_id": "uuid" }`

`POST /chat-rate`

Submits like/dislike after a chat.

Request:

```
json
{
  "chat_id": "uuid",
  "rating": "like", // or "dislike" or "skipped"
  "comment": "Great chat!"
}
```

6.5 Female chat request response

POST /female-chat-request-respond

Female accepts or declines an incoming request.

Request:

```
json

{
  "request_id": "uuid",
  "action": "accept"    // or "decline"
}
```

Response on accept: `{ "chat_id": "uuid" }` (creates a `chats` row, returns the id for Phase 2 chat screen).

6.6 Notifications

Method	Endpoint	Purpose
GET	<code>/rest/v1/notifications?user_id=eq.{me}&order=created_at.desc</code>	list
PATCH	<code>/rest/v1/notifications?id=eq.{id}</code>	mark read
PATCH	bulk update	mark all read

POST /notification-fcm-token-register

Registers/updates the user's FCM device token.

Request:

```
json

{
  "token": "fcm-token-string",
  "platform": "android"
}
```

6.7 Block / Report

POST /block-user

Request: { "blocked_id": "uuid" }

POST /report-user

Request:

```
json
{
  "reported_id": "uuid",
  "reason": "harassment",
  "comment": "optional",
  "chat_id": "uuid (optional)"
}
```

POST /support-report-issue

Request:

```
json
{
  "category": "payment",
  "description": "...",
  "screenshot_url": "(optional Cloudinary URL)"
}
```

6.8 Realtime channels (Supabase Realtime)

Used for live in-app updates without polling.

Channel	Subscribed by	Triggers on
chat_requests:female_id=eq.{me}	female	new INSERT (new incoming request)
chat_requests:male_id=eq.{me}	male	UPDATE to status (accept/decline)
notifications:user_id=eq.{me}	both	INSERT
female_profiles:user_id=eq.{me} {watched_id}	male	UPDATE to is_online (favourite went online)

7. Environment Variables

```
bash

# .env.example

# Supabase
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...      # backend only, NEVER expose to client

# MSG91
MSG91_AUTH_KEY=...
MSG91_TEMPLATE_ID=...
MSG91_SENDER_ID=...

# Cloudinary
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...          # backend only

# Razorpay
RAZORPAY_KEY_ID=...
RAZORPAY_KEY_SECRET=...             # backend only
RAZORPAY_WEBHOOK_SECRET=...

# Firebase (FCM)
FCM_SERVER_KEY=...                  # backend only
# Client uses google-services.json / GoogleService-Info.plist

# App
APP_ENV=development                 # or staging / production
SUPPORT_EMAIL=support@yourapp.com
```

In Flutter, only the non-secret keys (`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `RAZORPAY_KEY_ID`, `CLOUDINARY_CLOUD_NAME`) are bundled. All secrets stay in Supabase Edge Function environment variables.

8. Auth Flow (detailed)

Signup (Female)

1. User enters Name, Age, Gender, Password, Confirm, Mobile → taps "Generate OTP"

2. Client calls `POST /auth-otp-send` with `{purpose: signup}`
3. Server creates `otp_codes` row, calls MSG91 to deliver SMS
4. User receives SMS, enters 6-digit code
5. Client calls `POST /auth-otp-verify` → returns short-lived token
6. Client calls `POST /auth-signup-complete` with full profile data + the token → creates `users` + `female_profiles` rows, returns session
7. Bank details page (skippable) → PATCH on `bank_details`
8. Verification photo upload → `POST /upload-sign` → Cloudinary upload → PATCH `female_profiles.verification_photo_url + verification_status = pending`
9. Show "Verification submitted" → route to Login

Login (Female)

1. User enters phone → "Continue"
2. Client calls `POST /auth-login-check` → returns `verification_status`
3. If `pending`: show popup, stay on phone screen
4. If `missing`: route to face capture flow
5. If `verified`: show password screen
6. User enters password → standard Supabase Auth `signInWithPassword`
7. On success: session created, redirect to Home

Signup / Login (Male)

Same OTP flow, no verification gate. Login goes directly to phone+password screen.

Forgot Password

1. Phone → `POST /auth-forgot-password-send`
2. OTP entered → `POST /auth-forgot-password-verify` → returns `reset_token` (5 min)
3. New password → `POST /auth-forgot-password-reset` with token

9. Key Integrations

Cloudinary

- All client uploads use **signed uploads**. Client calls `POST /upload-sign` to get a signature, then uploads directly to `https://api.cloudinary.com/v1_1/{cloud_name}/auto/upload` with the signature. The API secret never leaves the server.
- Folders: `profile_pics/`, `verification_photos/`, `chat_media/` (Phase 2), `reports/`
- Verification photos: store with `{access_mode: authenticated}` (requires signed URL to view) and configure auto-delete after dispute window.
- Use auto-format and auto-quality transformations.

MSG91

- Use the Send OTP API or the standard SMS API with a DLT-approved template.
- Server stores the *hash* of the OTP in `otp_codes.code_hash` (use bcrypt or similar). Never store raw codes.
- Rate limit: max 3 OTPs per phone per 15 minutes; max 5 verification attempts per OTP.

Razorpay

- Coin purchase: `male-payment-create` creates an order server-side, returns `order_id` to client. Client opens Razorpay Checkout SDK. On success, client posts the payment details to `male-payment-verify` which validates the HMAC signature server-side before crediting coins.
- Webhook (`/razorpay-webhook` Edge Function) handles delayed events (`payment.captured`, `payment.failed`).
- For payouts (admin side): RazorpayX Payouts API used by admin dashboard, not the mobile app.

Firebase Cloud Messaging

- Client registers device token via `POST /notification-fcm-token-register` after login.
 - Server-side: when a new `chat_request` is created (for female) or a `chat_request.status` updates to `accepted/declined` (for male), a DB trigger calls a webhook that sends an FCM push via the `fcm.ts` shared helper.
 - Topic-based push not needed; all messages are user-targeted.
-

10. Row Level Security (RLS) Strategy

Default: **all tables have RLS enabled**. Policies are restrictive by default and granted explicitly.

Pattern examples:

- `users`: SELECT permitted for `auth.uid() = id` (own row) and for any authenticated user reading the limited public fields via the `available_females_view`.
- `female_profiles`: SELECT own row, UPDATE own row. No INSERT/DELETE from clients.
- `male_profiles`: SELECT/UPDATE own row.
- `transactions`: SELECT own rows only. INSERT only via Edge Function (service role bypasses RLS).
- `payouts`: SELECT own rows (female), INSERT via Edge Function only.
- `chats`: SELECT where `male_id = auth.uid() OR female_id = auth.uid()`.
- `chat_messages` (Phase 2): SELECT only by participants.

- `bank_details`: SELECT/UPDATE own row.
- `ratings`: INSERT only via Edge Function; SELECT own ratings.

All sensitive mutations route through Edge Functions running with the service-role key, which bypass RLS — but the function itself validates `auth.uid()` against the resource it's modifying.

11. Development Setup

One-time setup

```
bash

# Install Supabase CLI
brew install supabase/tap/supabase # or scoop / npm

# Initialize Supabase project
cd supabase
supabase init
supabase login
supabase link --project-ref <project-id>

# Push migrations
supabase db push

# Deploy edge functions
supabase functions deploy auth-otp-send
supabase functions deploy auth-otp-verify
# ... etc, or:
supabase functions deploy --no-verify-jwt

# Set function secrets
supabase secrets set MSG91_AUTH_KEY=xxx
supabase secrets set CLOUDINARY_API_SECRET=xxx
supabase secrets set RAZORPAY_KEY_SECRET=xxx
# ... etc
```

Flutter setup

```
bash
```

```
cd mobile
flutter pub get
# Generate riverpod / freezed files
flutter pub run build_runner build --delete-conflicting-outputs

# Run on device
flutter run
```

Folder conventions

- Every screen filename ends with `_screen.dart`.
 - Every dialog filename ends with `_dialog.dart`.
 - Every reusable widget filename is descriptive (`_primary_button.dart`).
 - Every provider filename ends with `_provider.dart` or `_notifier.dart`.
 - Every repository filename ends with `_repository.dart`.
-

12. Open Items / Phase 2

Not designed yet, but the schema supports them:

- Active chat screen (text/image/video messages) — uses `chat_messages` table + Supabase Realtime channel per chat
 - Queue management (5-person queue with 20-min timeout) — uses `chat_requests.queue_position` + scheduled function for timeouts
 - Coin deduction during active chat — periodic Edge Function or client-driven tick
 - Admin dashboard (web app, separate Next.js project; uses same Supabase backend)
-

13. AI Coding Assistant Guidance

When generating code with Cursor / Claude Code, ask it to:

1. Read this file first as project context.
2. Follow the folder structure in Section 3 strictly.
3. Use Riverpod for state, `go_router` for routing, Dio for HTTP, Supabase client for Supabase calls.
4. Use freezed for model classes (immutable, `copyWith`, JSON serialization).
5. Never put business logic in widgets — always extract to providers/notifiers.
6. Write the corresponding test alongside any new feature.
7. Reference `docs/api-reference.md` for backend contract.